02 사이킷런으로 시작하는 머신러닝 & 03 평가

02 사이킷런으로 시작하는 머신러닝

분류(지도학습 : Supervised Learning)

지도학습 : 모델 학습 후 예측

sklearn의 규칙

- sklearn.tree : 의사결정트리 같은 트리 기반 머신러닝 알고리즘
- sklearn.model_selection : 데이터 분할과 하이퍼파라미터 튜닝을 위한 모듈

학습 데이터와 테스트 데이터로 분리하는 데는 train_test_split()

피처(Feature) 데이터셋 : 설명 변수들(X)
레이블(Label) 데이터셋 : 예측하려는 정답 값(y)

ex. **X_train, X_test, y_train, y_test = train_test_split(iris_data, iris_label, test_size=0.2, random_state=11)**

- 선택 파라미터 : text_size, train_sizem shuffle, random_state, train_test_split()

*의사결정트리(Decision Tree) : 데이터를 질문으로 계속 나누면서 분류 및 예측

1. 데이터 나누기
2. 모델 학습 (의사결정트리 형성)
3. 예측
4. 평가

```
#데이터 세트 분리

X_train, X_test, y_train, y_test = train_test_split(iris_data, iris_label,
test_size=0.2, random_state=11)
```

```
dt_clf = DecisionTreeClassifier(random_state=11)
```

```
#모델 학습

dt_clf.fit(X_train, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(random_state=11)
```

```
# 예측 수행

pred = dt_clf.predict(X_test)
```

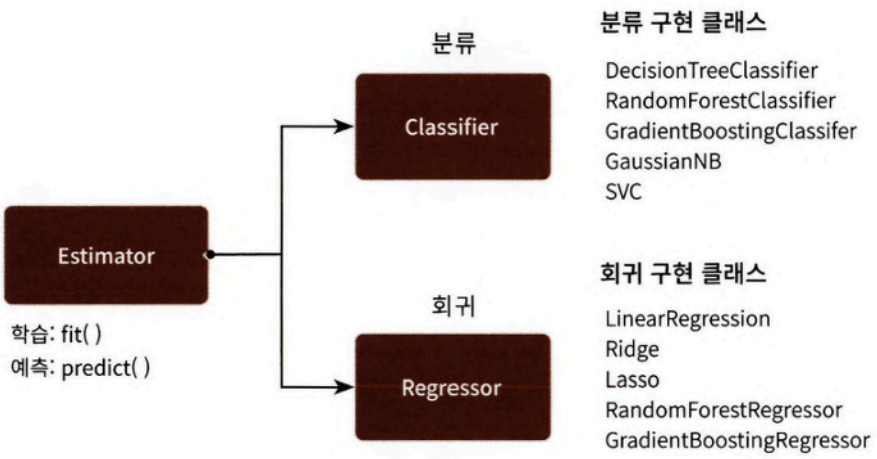
```
# 평가

from sklearn.metrics import accuracy_score
print('예측 정확도: {0:.4f}'.format(accuracy_score(y_test, pred)))
```

```
예측 정확도: 0.9333
```

02-3 사이킷런의 기반 프레임워크 익히기

지도학습엔 크게 분류와 회귀, 학습 fit과 예측 pred를 이용



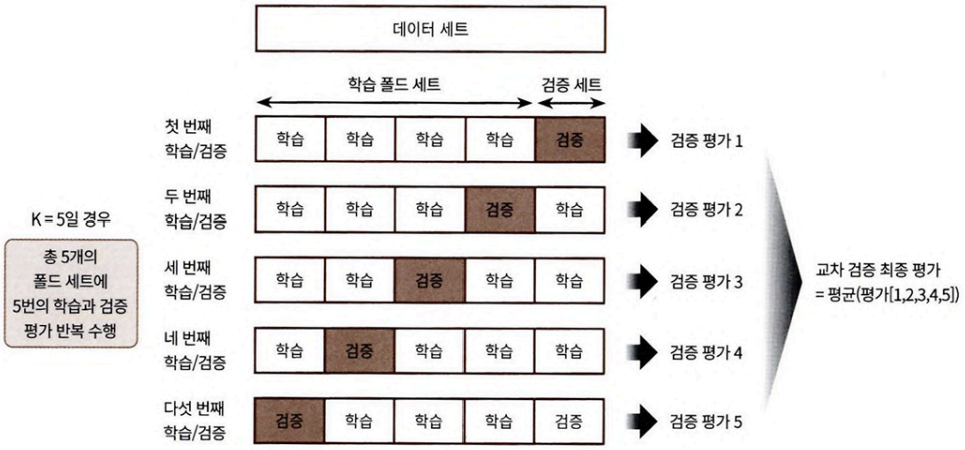
파이썬 내의 데이터는 대부분 딕셔너리 형태 → 키로 쉽게 꺼낼 수 있음.

02-4 Model Selection 모듈 소개

overfitting을 막기 위한 교차검증

1. 폴드 세트를 설정
2. for 루프에서 반복으로 학습 및 테스트 데이터의 인덱스를 추출
3. 반복적으로 학습과 예측을 수행하고 예측 성능을 반환

- K 폴드 교차 검증



데이터를 K개로 나누어 각각 다르게 test set을 설정하고 K번 검증 후 평균

- Stratified K 폴드

데이터가 비율이 맞지 않을 경우(혹은 이상치가 있을 경우 (ex. 주택 가격 중 사기 데이터)) K 폴드는 문제가 생김. 이상치를 고려한 후 나눈 뒤 학습하는 검증.

ex. A그룹 데이터 20개, B 그룹 데이터 1784개 → 분할이 안 됨

그렇기에 Stratified K 폴드는 분류 모형에서만 쓰고, 회귀 모형에서는 사용하지 않음! (K폴드는 레이블에 상관 없이 나누기에 회귀도 가능)

- cross_val_score() - 교차 검증과 최적 하이퍼 파라미터 튜닝을 한 번에

Kfold를 편리하게 수행하는 API

ex. 성능 지표는 정확도(accuracy), 교차 검증 세트는 3개

scores = cross_val_score(dt_clf, data, label, scoring='accuracy',cv=3)

- GridSearchCV

최적의 파라미터 (하이퍼 파라미터의 최적 값)을 찾게 해줌; 내가 설정해야 하는 파라미터 max_depth 같은 변수의 최적 값을 알려줌)

estimator: 사용할 모델 (classifier, regressor, pipeline 등)

param_grid: 튜닝할 하이퍼파라미터와 후보 값들을 담은 딕셔너리

scoring: 모델 성능 평가 기준 (ex. accuracy)

cv: 교차검증에서 데이터를 나눌 fold 수

refit: 최적의 하이퍼파라미터로 모델을 다시 학습할지 여부 (기본값 True)

1. GridSearchCV.fit(학습데이터)을 통해 cv(교차검증 fold)로 나누고 데이터 학습
2. cv_results_ 속성에 저장
3. cv_results_는 딕셔너리 형태이며 key = 항목 이름, value = 결과 리스트로 구성

02-05 데이터 전처리

레이블 인코딩(Label encoding) : LabelEncoder

각각의 라벨에 번호를 부여하는 방식

가중치가 부과될 수 있음

원-핫 인코딩(One-Hot Encoding) : OneHotEncoder

새로운 열을 만들고 포함 여부에 따라 0, 1로 나눔

원핫은 문자열을 숫자형으로 변환해야 하지만
pd.get_dummies()

를 이용하면 바로 변환할 수 있음.

피처 스케일링과 정규화

표준화 (Standardization)

$$x_i_new = \frac{x_i - mean(x)}{stdev(x)}$$

평균 0, 분산 1로 표준화 하는 과정, 가우시안 정규 분포

StandardScaler()

정규화(Normalization)

$$x_i_new = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

모든 값을 0~1 사이로 만드는 과정

MinMaxScaler

데이터값을 0~1 사이로 변환, 데이터 분포가 가우시안 분포가 아닐 경우에 적용함

학습 데이터와 테스트 데이터의 스케일링 변환 시 유의점

1. 학습 데이터: fit() → transform() (또는 fit_transform())
2. 테스트 데이터: 절대 fit() 금지, 오직 transform()만 사용
3. 이유: 학습 데이터 기준으로 테스트 데이터를 변환해야 모델 예측이 일관됨

ㄴ 학습 데이터는 transform의 형태로만 끝나기 때문

03 평가 (분류 위주)

- 정확도(Accuracy)
- 오차행렬(Confusion Matrix)
- 정밀도(Precision)
- 재현율(Recall)
- F1 스코어
- ROC AUC

정확도(Accuracy)

$$\text{정확도(Accuracy)} = \frac{\text{예측 결과가 동일한 데이터 건수}}{\text{전체 예측 데이터 건수}}$$

그러나 정확도의 경우 데이터가 불균형한 이진 분류 시 정확도가 왜곡될 수 있음 (ex. 주사위가 6인 경우 0.16 혹은 0.84이기에 0.84로 찍는 게 유리함)

오차행렬(Confusion Matrix)

		예측 클래스 (Predicted Class)	
		Negative(0)	Positive(1)
실제 클래스 (Actual Class)	Negative(0)	**TN** (True Negative)	**FP** (False Positive)
	Positive(1)	**FN** (False Negative)	**TP** (True Positive)

정확도 = 예측 결과와 실제 값이 동일한 건수/전체 데이터 수 = (TN + TP)/(TN + FP + FN + TP)

성능의 여러 면모를 파악 할 수 있으나 TN + TP 중 한 값은 높고 한 값은 낮아도 정확도가 올라가는 경우가 발생하기도 함.

정밀도(Precision)와 재현율(Recall)

- **정밀도(Precision)**: Positive로 예측한 것 중 실제로 Positive인 비율

- **재현율(Recall)**: 실제 Positive 중에서 모델이 올바르게 Positive로 예측한 비율

1️⃣ **정밀도(Precision)**

- **정의**: 모델이 Positive로 예측한 것 중 실제 Positive 비율

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **중점**: FP(실제 Negative를 Positive로 잘못 예측한 수)를 줄이는 데 초점
- **활용 상황**: Negative를 Positive로 잘못 판단했을 때 업무상 피해가 큰 경우
  - 예: 스팸메일 필터링에서 정상 메일을 스팸으로 분류하면 사용자 불편

2️⃣ **재현율(Recall, 민감도, TPR)**

- **정의**: 실제 Positive 중 모델이 올바르게 Positive로 예측한 비율

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **중점**: FN(실제 Positive를 Negative로 잘못 예측한 수)을 줄이는 데 초점
- **활용 상황**: Positive를 놓쳤을 때 큰 영향이 발생하는 경우
  - 예: 암 진단, 금융 사기 탐지

정밀도와 재현율의 트레이드오프

정밀도와 재현율은 상호 보완적인 평가 지표이기에,

한 쪽이 올라가면 한 쪽은 내려가는 트레이드오프가 발생

predict() : 개별 데이터로 예측 확률 반환

→ Binarize() : 예측 값이 더 큰 class 반환

→ get_clf_eval() : predict(), predict_proba()에 기반한 평가 지표

!! 임계값을 조정하여 정밀도/재현율 조정 가능

임계값? Positive 판단할 비율 지표; ex. 임계값 0.4 ⇒ 확률 0.4 이상부터 Postive

임계값 ⬇️ → Positive ⬆️→ 정밀도⬆️ (양성 예측이 많아졌기에)

- get__eval_by_threshold() 함수로 임계값을 조절하며 베스트 찾기
- precision_recall_curve()로 정밀도와 재현율 임계값에 따른 값 변화를 곡선 그래프로 시각화 가능
- 임계값 ⬆️ → 재현율 ⬇️, 정밀도⬆️

03-04 F1 스코어

$$F1 = \frac{2}{\frac{1}{recall} + \frac{1}{precision}} = 2 * \frac{precision * recall}{precision + recall}$$

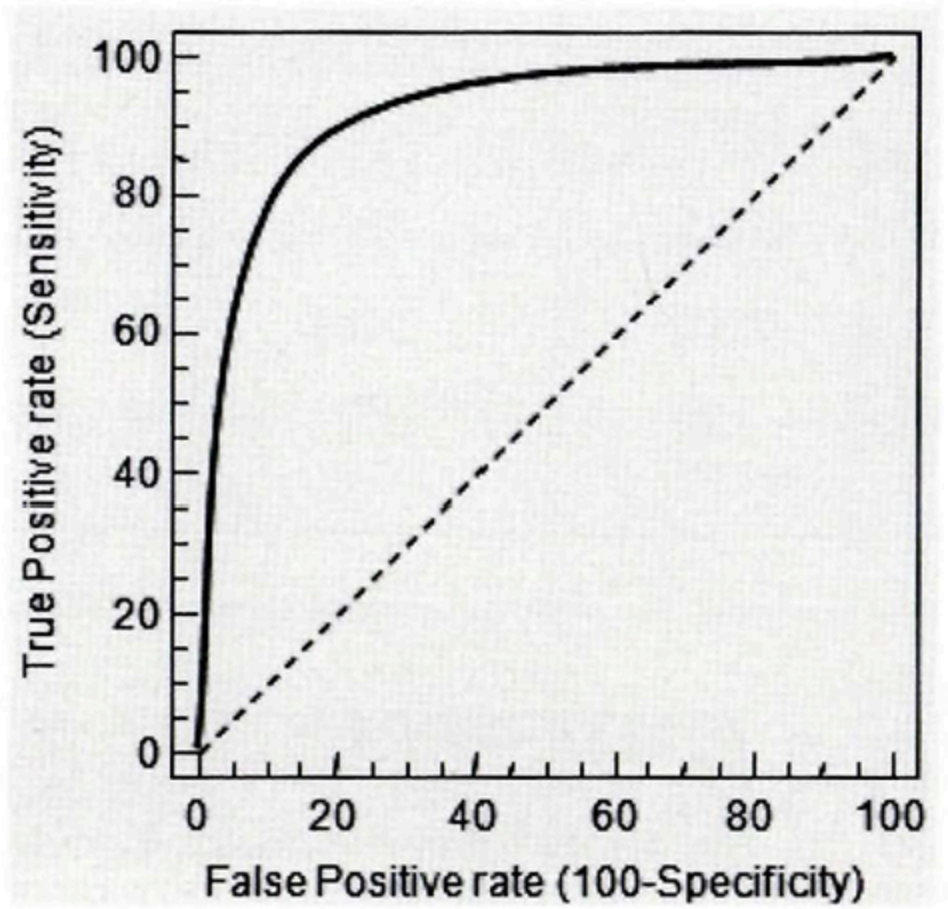
정밀도와 재현율에 큰 차이가 없을 때 우수한 값을 가짐

03-05 ROC 곡선과 AUC

TPR(True Positive Rate) : 재현율 = 민감도 ↔ TNR(True Negative Rate) : 특이성

FPR = FP / (FP + TN) = 1 - TNR = 1- 특이성

ROC 곡선 : FPR(False Positive Rate, X축)이 변할 때 TPR(True Positive Rate, Y축)의 변화 곡선



〈 ROC 곡선 예시 〉

직선에 가까울수록 성능이 떨어지며, 멀어질수록 성능이 뛰어난 것.